CS6230 CAD for VLSI Project Report

Viterbi Decoder: Design, Verification & Synthesis

Submitted by:

CS25D007 Rahul Anilkumar

EE25M092 Aashim Sikka

Contents

1 Introduction **3**

2 Approach towards the Design **3**

3 Design With BlueSpec Verilog **4**
3.1 Architecture Block Diagram 4
3.2 Finite State Machine 4
3.3 Interface 6

4 Verification Methodology **7**
4.1 Directed Testcases 7
4.2 Constrained Random Testcases 7
4.3 Negative Testcases 7
4.4 Automated Verification Framework 8
4.5 Bugs Identified During Verification 10

5 Synthesis Reports **10**
5.1 For Baseline Design 10
5.1.1 Synthesis reports of Baseline Design 10
5.2 Improvements Made in the Design 11
5.2.1 Timing Optimization 11
5.2.2 Area Optimization 11
5.2.3 Power Optimization 12
5.3 For Final Design 12
5.3.1 Synthesis Reports of Final Design 12

1 Introduction

The Viterbi algorithm is a dynamic programming technique used to determine the most probable sequence of hidden states, given a sequence of observations [1]. It is widely used with **Hidden Markov Models (HMMs)**, where the internal states of the system are not directly observable, and only the sequence of outputs generated by these hidden states is known. The primary objective of a Viterbi decoder is to analyze the observed sequence and compute the single most likely sequence of hidden states that could have produced it. The algorithm achieves this using state transition and emission probabilities in the forward path and performs backtracking from the final state to the initial state to identify the optimal state sequence.

2 Approach towards the Design

Before beginning the implementation, we analyzed the problem and defined the initial design strategy. The key assumptions and decisions made during the planning stage are listed below:

- We first determined the maximum possible values of N and M, and accordingly defined the ranges for `A.data`, `B.data`, and the intermediate arrays.
- Initially, `A.data` and `B.data` were thought as 2D arrays for easy understanding, and a pseudo-code was developed. Later, we made changes in our pseudo-code by considering them as 1D arrays, and the indexes were modified based on N and M accordingly.
- For storing state probabilities, the first approach was to store probabilities for all stages. Later, we realized that only the **current** and **previous** stage probabilities are required at any time. Hence, we optimized the design accordingly.
- The Floating Point Adder (FpAdder) was a critical design constraint. Since the problem involved only negative values, we implemented a positive floating point adder and manually forced the sign bit (MSB) to `1'b1`. This approach functioned correctly and was integrated into the design.
- The FpAdder requires a 24-bit mantissa addition. A Ripple Carry Adder (RCA) would not meet timing requirements, so based on prior knowledge, we selected the **Kogge-Stone Adder (KSA)** due to its $O(\log_2 n)$ delay. Later, based on synthesis results, we transitioned to **Brent–Kung Adder (BKA)**, another parallel prefix adder that consumes less area but more delay.
- The traceback logic was initially misunderstood—we assumed that a single previous state would map to all next states. However, after manually verifying test cases, we realized that a full traceback from the final state to the initial state is necessary. The design was updated accordingly to implement proper backtracking.

3 Design With BlueSpec Verilog

3.1 Architecture Block Diagram

Based on the specifications provided, the following architecture was finalized:

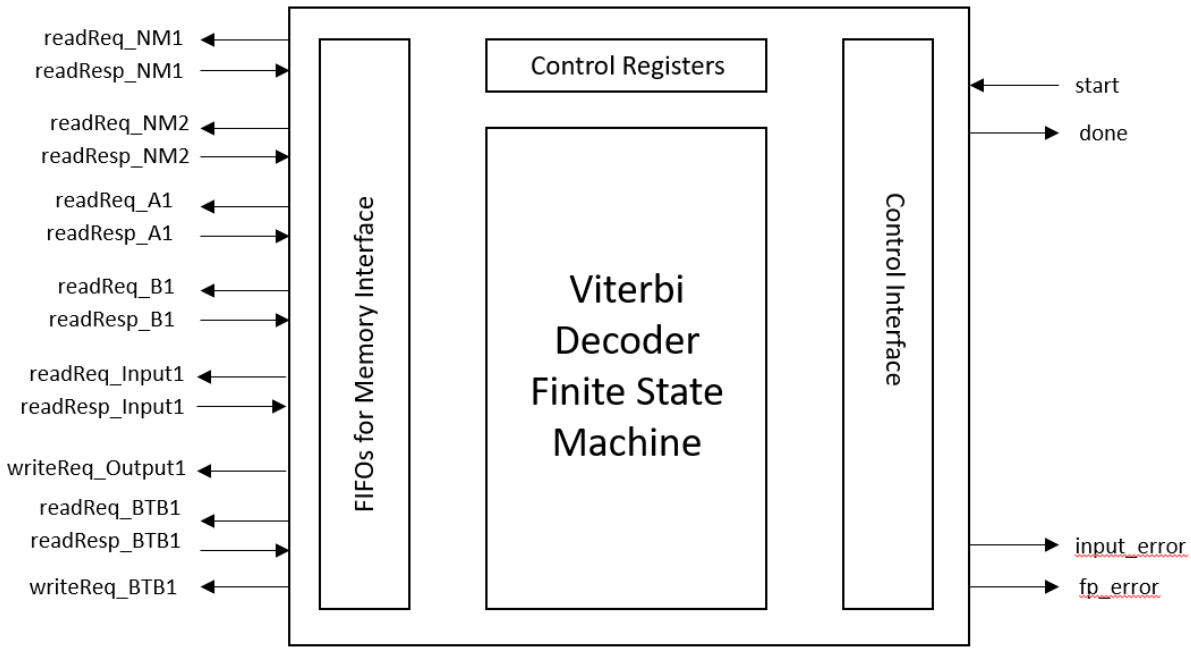

Figure 1: Viterbi Decoder Block Diagram

3.2 Finite State Machine

FSM was designed with the following states to execute Viterbi Algorithm, including Forward Pass and Traceback (**48 states**):

```
// Start States
IDLE,
START,

// Get N & M
INIT_GET_NM,
DONE_GET_NM,

// V1 Calculation
V1_CALC_START,
GET_INPUT_V1,
INIT_COMPUTE_V1,
```

```
LOOP_V1_INIT_GET_AB,
LOOP_V1_DONE_GET_AB,
LOOP_V1_INIT_FP_ADD,
LOOP_V1_DONE_FP_ADD,
LOOP_V1_NEXT_STATE,
V1_CALC_END,

// Vt Recursive Calculation (iterate till end)
VT_CALC_START,
VT_GET_INPUT,
VT_INIT_J_LOOP, // j loop from spec
VT_INIT_I_LOOP, // i loop from spec
VT_GET_PREV_PROB,
VT_INIT_GET_AB,
VT_DONE_GET_AB,
VT_INIT_FP_ADD, // fires prob + A
VT_DONE_FP_ADD,
VT_LOOP_I_LOOP, // Optimised for single-B addition
VT_SAVE_STATE,
VT_GET_B_AND_ADD, // Optimised for single-B addition
VT_ADD_B_TO_MAX,  // Optimised for single-B addition
VT_STORE_RESULT,  // Optimised for single-B addition
VT_STORE_BTB,
VT_LOOP_J_LOOP,

// Optimised :: Min reduction
VT_MIN_INIT,
VT_MIN_READ,
VT_MIN_WAIT,
VT_MIN_DONE,

// Memory access states for vCurr/vPrev
V1_WRITE_VCURR,
VT_WRITE_VCURR,
VT_READ_VPREV,
VT_READ_VPREV_WAIT,
SWAP_INIT,
SWAP_READ_VCURR,
SWAP_WRITE_VPREV,

// Traceback & Write output probable state
TRACEBACK_CALC_START,
TRACEBACK_LOOP_START,
TRACEBACK_LOOP_EXEC,
TRACEBACK_WRITE_FINAL_PROB,
```

```
// Finish States
WRITE_FF_MARKER,
DONE,
INPUT_ERROR,
FP_ERROR
```

3.3 Interface

Signal Name	Direction	Description
System Control		
1. start	Input	From testbench to start operation
2. done	Output	To testbench to indicate end of operation
Error Indicators		
1. inputError	Output	To testbench to indicate error in input
2. fpError	Output	To testbench to indicate FP calculation error
Memory Access		
1. readReq_NM1	Output	Read request to testbench for N value
2. readResp_NM1	Input	Read response from testbench with N value
3. readReq_NM2	Output	Read request to testbench for M value
4. readResp_NM2	Input	Read response from testbench with M value
5. readReq_A1	Output	Read request to testbench for transition probability
6. readResp_A1	Input	Read response from testbench with transition probability
7. readReq_B1	Output	Read request to testbench for emission probability
8. readResp_B1	Input	Read response from testbench with emission probability
9. readReq_Input1	Output	Read request to testbench for observation
10. readResp_Input1	Input	Read response from testbench with observation
11. WriteResp_Output1	Output	Write request to testbench with output values
12. readReq_BTB1	Output	Read request to testbench for Back Trace Buffer output
13. readResp_BTB1	Input	Read response from testbench with Back Trace Buffer output
14. WriteResp_BTB1	Output	Write request to testbench with Back Trace Buffer input

Table 1: Interface ViterbiDecoderIfc

4 Verification Methodology

For verifying the design, we devised a mix of directed, constrained, random and negative testcases. An automated framework was established using Shell script to run and verify testcases against a golden output generated by a Python script.

4.1 Directed Testcases

These tests are used to check functional correctness for interesting, valid input scenarios, and corner cases. The details are provided in the table below:

No.	Test Name	Description
1	`test_Small`	Testcase provided by TAs to do sanity checks
2	`test_Long`	Testcase provided by TAs with long sequences
3	`test_NM24`	Sanity testcase added for small N, M values
4	`test_NLarge`	Testcase for large N values
5	`test_MLarge`	Testcase for large M values
6	`test_N1M1F`	Corner case test for N = 1, M = 31
7	`test_N1FM1`	Corner case test for N = 31, M = 1

Table 2: Functional and Sanity Test Scenarios

4.2 Constrained Random Testcases

A Python script was used to generate random, valid scenarios with different values for N, M, transition, emission probabilities and input sequences. We decided to keep **20 testcases** to balance between sufficient randomization and test runtime. Six long testcases, each testing maximum values for input sequences when N = 1,2,3 and fourteen shorter tests with random probabilities and input sequence length were generated.

Constrained random verification **brought out a critical bug showing a one-off error**. For input sequence lengths of max valid length, the design was initially reporting an invalid sequence length. On debug, this was found to be due to an incorrect timeStep assignment. Subsequently, a further cornercase to tackle max input sequence length of 1022 (which would result in output length 1025) was added, which uncovered another bug limiting output address width to 10 bits.

4.3 Negative Testcases

It is also important to check the design behaviour when exposed to invalid input scenarios. Negative testing was also carried out after identifying possible invalid sequences in the input. Following are the negative cases tested:

No.	Test Name	Description
1	NEGATIVE_test_incorrect_N	Incorrect (> 31) value for number of states
2	NEGATIVE_test_incorrect_M	Incorrect (> 31) value for number of observations
3	NEGATIVE_test_incorrect_A_prob	Transition probability is given positive (invalid) values
4	NEGATIVE_test_incorrect_B_prob	Emission probability is given positive (invalid) values
5	NEGATIVE_test_incorrect_Input	Incorrect input (observation >M) is provided to design
6	NEGATIVE_test_N_0	Edge case with N = 0
7	NEGATIVE_test_M_0	Edge case with M = 0
8	NEGATIVE_test_Input_0	Edge case with Input = 0
9	NEGATIVE_test_N1M1F_1023_input	Incorrect Sequence length test
10	NEGATIVE_test_fpError	Test for FP error verification

Table 3: Negative Test Scenarios

The negative test case checks are designed to PASS the test if DUT reports an error. Screenshot from the negative test run is shown below:

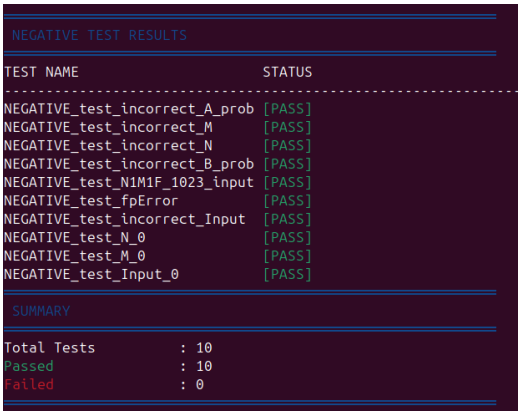

Figure 2: Verification Results for Negative Testcases (PASS indicates Error from DUT)

4.4 Automated Verification Framework

To enable automated verification and reproducibility, and automated framework was developed using Shell scripts. The setup allows to run individual testcases, or all tests (positive and negative separately), and report the results by comparing the outputs against the golden output generated by the reference script.

Results from the verification runs are shown below:

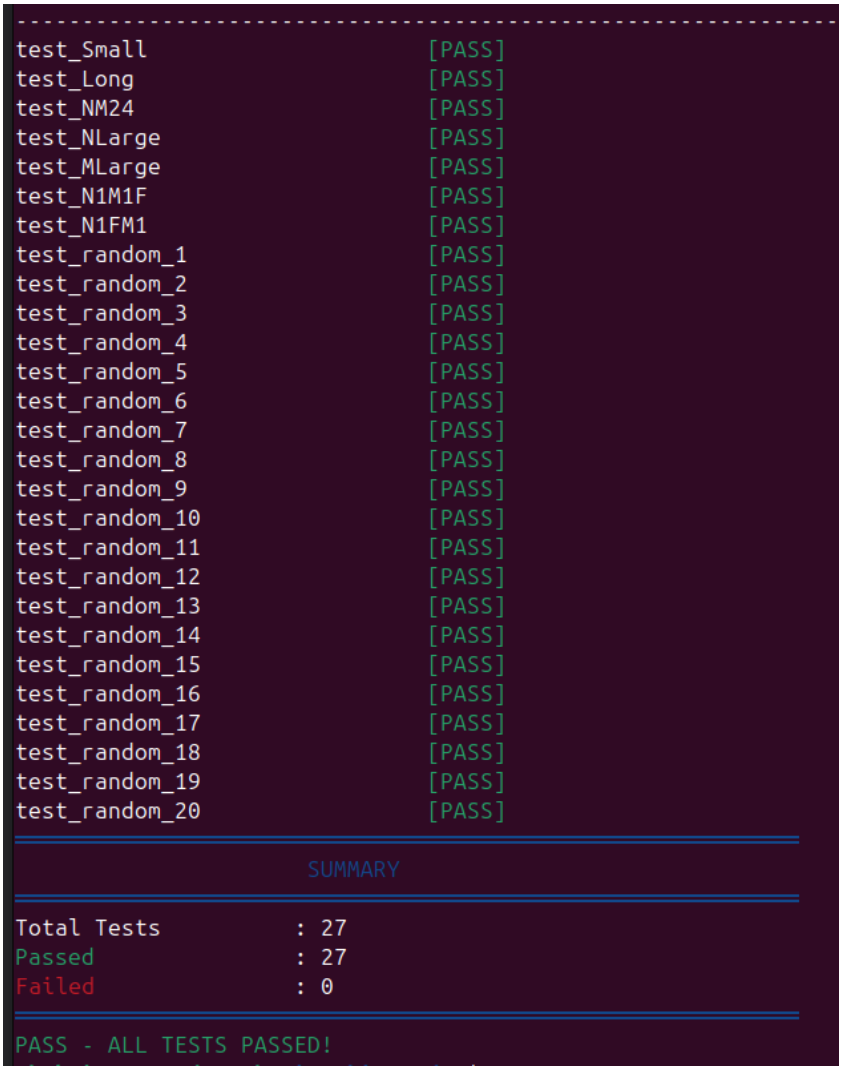

Figure 3: Verification Results for Positive and Constrained Random Testcases

4.5 Bugs Identified During Verification

No.	Bug Description	Resolution
1	No checks for N, M, or input sequence length	Updated RTL to check for inconsistencies
2	Incorrect rounding in FP Adder	Corrected after update in Forum
3	One-off error for long input sequences	Corrected `timeStep` in RTL
4	Incorrect address width for Output memory	Output address width to be 11 bits for 1022 input seq (1025 lines req).
5	Incorrect output file formatting	Output state in FSM updated
6	FSM looping incorrectly	FSM logic corrected
7	Unnecessary FP additions (Performance loss)	Optimised addition logic

Table 4: Bugs Identified During Verification and Their Fixes

5 Synthesis Reports

Maximum Clock Frequency = 0.253 GHz

5.1 For Baseline Design

A baseline design was first made with BlueSpec Verilog that is functionally correct, but not optimised for area, power or performance. Initial verification was done with Small and Long testcases supplied.

5.1.1 Synthesis reports of Baseline Design

On Synthesis, the Baseline gave these results for the targeted **Time = 6ns**.

```
clock CLK (rise edge)                                    6.000      6.000
clock network delay (ideal)                              0.000      6.000
clock uncertainty                                       -1.000      5.000
finalMaxProb_reg_2_/CLK (DFFX1_LVT)                      0.000      5.000 r
library setup time                                      -0.069      4.931
data required time                                                  4.931
--------------------------------------------------------------------------
data required time                                                  4.931
data arrival time                                                 -26.467
--------------------------------------------------------------------------
slack (VIOLATED)                                                  -21.536
```

Figure 4: Baseline Slack

```
Area
-----------------------------------
Combinational Area:    50524.844490
Noncombinational Area: 57727.286469
Buf/Inv Area:           6192.472749
Total Buffer Area:           763.45
Total Inverter Area:        5429.02
Macro/Black Box Area:      0.000000
Net Area:                42083.654001
-----------------------------------
Cell Area:             108252.130959
Design Area:           150335.784960
```

Figure 5: Baseline Area

```
--------------------------------------------------------------------------------
                                    Switch   Int      Leak     Total
Hierarchy                           Power    Power    Power    Power    %
--------------------------------------------------------------------------------
mkViterbiDecoder                     218.316  903.087 2.64e+09 3.77e+03 100.0
  writeReqFIFO_Output (FIFO2_0000002a_1)
                                         0.283    1.573 1.95e+07   21.384   0.6
```

Figure 6: Baseline Power

- **Minimum Time Period** = 27.536 ns
- **Maximum Clock Frequency** = 36.316 MHz
- **Area**(at 6ns) = $150.335 \times 10^3\,\mu m^2$
- **Power**(at 6ns) = 3770 µW

5.2 Improvements Made in the Design

5.2.1 Timing Optimization

After analyzing the critical path from the baseline design reports, we observed that the **minimum reduction loop** contributed the maximum delay, as it performed **32 parallel comparisons** in a single cycle. To improve timing performance, we initially introduced **4-stage pipelining**, which reduced the delay and achieved a timing of **6.9 ns**. However, this significantly increased the overall area. Later, after performing area optimization (described in the next section), we observed that the required timing constraints were already satisfied without pipelining. Therefore, the pipelining stages were removed to maintain an area-efficient design while still meeting the timing target.

5.2.2 Area Optimization

Initially, it was unclear which module was consuming the most area, so multiple strategies were explored to reduce it:

1. In the **FP Adder module**, we originally used a **Kogge-Stone Adder (KSA)**, which performs 32-bit addition in 5 gate delays but occupies a large area. It was replaced with a **Brent-Kung Adder (BKA)**, which has a higher delay of **9 gate delays** but uses approximately **one-third the area** of KSA. This resulted in moderate area savings.

2. The **working memory was initially integrated inside the main design**. Moving it outside the core module reduced the total area by approximately **50%**, bringing it to approximately 50k, although it was still above the desired limit.

3. From the synthesis report, we identified that **FIFO2 contributed significantly to area overhead**. Replacing FIFO2 with **FIFO1** helped reduce the resource utilization further.

4. Further analysis revealed that the **minimum reduction loop instantiated 32 comparators in parallel**. This was redesigned to use a **single comparator sequentially**, which dramatically improved area efficiency. After this optimization, the total design area was reduced to approximately **15k**, successfully meeting the requirement.

5.2.3 Power Optimization

In the baseline design, power consumption exceeded the required limit. However, since **power is proportional to area**, priority was given to optimizing area and timing first. Once the area target was achieved, power consumption automatically fell within acceptable limits. Therefore, no further power-specific optimizations were required.

5.3 For Final Design

5.3.1 Synthesis Reports of Final Design

On Synthesis, the Final Design gave these results for the targeted **Time = 6ns**. Although for finding the maximum clock frequency, we synthesized for **Time = 3ns**, whose timing report is also attached.

```
clock CLK (rise edge)                                   6.000      6.000
clock network delay (ideal)                             0.000      6.000
clock uncertainty                                      -1.000      5.000
fpAddResult1_reg_24_/CLK (DFFX1_LVT)                    0.000      5.000 r
library setup time                                     -0.084      4.916
data required time                                                 4.916
-------------------------------------------------------------------------
data required time                                                 4.916
data arrival time                                                 -4.836
-------------------------------------------------------------------------
slack (MET)                                                        0.079
```

Figure 7: Final Timing Report

```
Area
--------------------------------
Combinational Area:     9402.819759
Noncombinational Area:  4872.448923
Buf/Inv Area:            898.399045
Total Buffer Area:            26.94
Total Inverter Area:         871.46
Macro/Black Box Area:       0.000000
Net Area:                2883.079555
--------------------------------
Cell Area:              14275.268682
Design Area:            17158.348237
```

Figure 8: Final Area

```
------------------------------------------------------------------------------
                                  Switch   Int      Leak     Total
Hierarchy                         Power    Power    Power    Power    %
------------------------------------------------------------------------------
mkViterbiDecoder                    31.074  194.306 3.23e+08  548.352 100.0
  writeReqFIFO_Output (FIFO1_0000002b_1)
                                     0.164    1.216 9.51e+06   10.886   2.0
  writeReqFIFO_BTB (FIFO1_0000002a_1)  9.30e-02    0.646 9.25e+06    9.987   1.8
  readRespFIFO_NM2 (FIFO1_00000020_1_0)
```

Figure 9: Final Power

For finding the maximum clock frequency, time was set to 3ns and simulated multiple times with different clock periods, and we found the minimum clock period as follows:

```
clock CLK (rise edge)                                    3.900      3.900
clock network delay (ideal)                              0.000      3.900
clock uncertainty                                       -1.000      2.900
fpAddResult1_reg_29_/CLK (DFFX1_LVT)                     0.000      2.900 r
library setup time                                      -0.083      2.817
data required time                                                  2.817
------------------------------------------------------------------------------
data required time                                                  2.817
data arrival time                                                  -2.887
------------------------------------------------------------------------------
slack (VIOLATED)                                                   -0.070
```

Figure 10: Final time report for CLK=3.9

```
clock CLK' (rise edge)                                   1.980      1.980
clock network delay (ideal)                              0.000      1.980
clock uncertainty                                       -1.000      0.980
clk_gate_inputValue_reg/latch/CLK (LATCHX1_LVT)          0.000      0.980 r
time borrowed from endpoint                              1.118      2.098
data required time                                                  2.098
------------------------------------------------------------------------------
data required time                                                  2.098
data arrival time                                                  -2.098
------------------------------------------------------------------------------
slack (MET)                                                         0.000
```

Figure 11: Final timing report for CLK=3.96

- **Minimum Time Period** = 3.96 ns

- **Maximum Clock Frequency** = 0.253 GHz
- **Area**(at 6ns) = $17.158 \times 10^3\,\mu m^2$
- **Power**(at 6ns) = 548.352 µW

References

[1] Wikipedia, "Viterbi Algorithm - Wikipedia." Accessed: 09-11-2025.